

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла):

студентка групи AI-241
Горбатова Вікторія
Юріївна

Перевірив:

ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	118	4
Час, с	4.77	0.43
Токенів/с	28.58	56.52

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	0 / 5
eval_count	1273.4	118.8
Час, с	55.45	2.29

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Я порівнював міркувальну модель (qwen3:4b) та інструктивну (llama3.2:3b) на простому фактичному питанні. Відповідь обидві дали однаково, але міркувальна зге нерувала в десятки разів більше токенів. Куди пішли ці токени?

Відповідь: У міркувальних моделях додаткові токени витрачаються на внутрішній ланцюжок роздумів. Навіть якщо підсумкова відповідь складається з одного слова («Київ»), модель спочатку аналізує запит, перевіряє факт, перевіряє виконання обмеження «відповідай одним словом» і лише потім формує остаточний результат.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?

Відповідь: міркувальні моделі дають ключову перевагу в складних математичних задачах, логічних головоломках, написанні складного коду та багатокроковому аналізі даних, де потрібна формалізація умов. Воно невиправдане для фактчекінгу, простих запитань «запит–відповідь», перекладу, класифікації чи форматування тексту, оскільки це призводить до суттєвого перевитрачання часу та ресурсів без підвищення якості.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: інструктивні моделі навчають переважно за допомогою SFT (Supervised Fine-Tuning) та RLHF на парах «запит — пряма відповідь» для точності виконання вказівок користувача. Міркувальні моделі додатково тренують за допомогою навчання з підкріпленням (RL) на кроках міркування, заохочуючи модель генерувати, перевіряти та коригувати свої проміжні гіпотези перед наданням фінального результату.

4. Чи означає більша кількість згенерованих токенів у міркувальній моделі, що вона «розумніша» за інструктивну?

Відповідь: ні, більша кількість токенів свідчить лише про використання іншого алгоритму генерації (розгортання внутрішнього ланцюжка CoT), а не про рівень «інтелекту». Якщо задачу можна розв'язати прямо, додаткова генерація є виключно накладними витратами обчислювальної потужності.

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b

Відповідь: різниця зумовлена перш за все розміром моделей (~4 млрд параметрів проти ~3 млрд) та оптимізацією архітектури/токенізатора. Більша кількість параметрів у qwen3:4b вимагає більше обчислень на один токен, що зменшує пропускну здатність (28.58 ток/с проти 56.52 ток/с у llama3.2:3b).

Висновок за завданням 1

Експеримент показав, що два показники — швидкість генерації та загальний час відповіді — не можна плутати. Перший характеризує обладнання й розмір моделі, другий — обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 11,1 раза за часом та у 29,5 раза за кількістю токенів. На задачі про ручку й зошит ситуація протилежна — саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 443,6 с проти 18,3 с, що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Для дослідження ефективності керування поведінкою моделей за допомогою системного контексту було проведено експеримент із порівняння двох підходів до формування system-повідомлення: загального ("лаконічний асистент") та жорстко обмеженого ("українською, 2 речення"). Тестування проводилося на інструктивній моделі llama3.2:3b та міркувальній (reasoning) моделі qwen3:4b при виконанні однакового користувацького запиту. Оцінювалися точність дотримання формальних обмежень, підсумкова довжина відповіді, мовна якість, а також витрати токенів (prompt_tokens, completion_tokens, total_tokens).

Таблиця 3: Вплив system повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	227	Некоректна українська	3	Ні, (галюцинація змісту)
llama3.2:3b	українською, 2 речення	214	Суміш української та англійської	3	Ні, (3 речення замість 2, суржик)

qwen3:4b	лаконічний асистент	382	Українська	4	Так
qwen3:4b	українською, 2 речення	208	Українська	2	Так

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	50	74	124
llama3.2:3b	українською, 2 речення	54	70	124
qwen3:4b	лаконічний асистент	53	2683	2736
qwen3:4b	українською, 2 речення	62	381	443

Відповіді на питання (Крок 6):

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: модель qwen3:4b точніше дотрималася інструкції, видавши чітко 2 речення. Модель llama3.2:3b згенерувала 3 речення та припустилася синтаксичних помилок.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: так, llama3.2:3b використала слово-зв'язку англійською мовою ("Meanwhile") та згенерувала псевдонауковий суржик, що свідчить про слабку підтримку української мови.

3. Як змінився prompt_tokens при зміні systemповідомлення і чому?

Відповідь: показник prompt_tokens зростає при зміні системного пром프트у у двох моделях (з 50 до 54 у Llama та з 53 до 62 у Qwen), оскільки нова системна інструкція виявилася довшою за кількістю символів і слів.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: це пояснюється тим, що qwen3:4b є міркувальною моделлю і спочатку генерує прихований ланцюжок роздумів (``), токени якого автоматично додаються до загальної суми completion_tokens.

Відповіді на питання (Крок 7):

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними systemповідомленнями. Чому моделі порізно дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: точність дотримання інструкцій залежить від обсягу навчального датасету (RLHF/SFT) та наявності фази міркування. qwen3:4b під час роздумів аналізує власні обмеження, тоді як llama3.2:3b генерує текст безперервно і частіше втрачає контекст обмежень.

2. Чим systemповідомлення відрізняється від userповідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: System-повідомлення задає системний контекст і передається через окремі синтаксичні теги токенизатора для визначення ролі. Проте у малих інструктивних моделях воно не завжди має абсолютний пріоритет і може «перебиватися» останнім user-запитом.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: міркувальна модель формує проміжні кроки (планування, перевірка вимог до мови та кількості речень), і всі ці додаткові токени мислення зараховуються до сумарного completion_tokens.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: більш зрілі та міркувальні моделі намагаються знайти компроміс або віддають перевагу system-правилам. Малі інструктивні моделі частіше виконують команду з user-повідомлення як більш свіжу в послідовності токенів.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: малі моделі мають обмежену кількість параметрів для підтримки менш поширених мов. При генерації технічних термінів їхній розподіл ймовірностей

збивається на більш домінуючу англійську або російську мову з навчальної вибірки.

Висновок за завданням 2

Експеримент показав, що вплив system-повідомлення суттєво залежить від архітектури моделі та наявності фази міркування. Інструктивна модель llama3.2:3b продемонструвала слабку стійкість до системних обмежень: при зміні пром프트 вона згенерувала 3 речення замість 2 та припустилася мовних помилок зі змішуванням мов, хоча обсяг total_tokens залишився однаковим (124). Натомість міркувальна модель qwen3:4b точно виконала всі вимоги щодо мови та ліміту в 2 речення, проте ціною цього стала суттєва перевитрата ресурсів — показник completion_tokens зріс до 381 за рахунок прихованого ланцюжка роздумів. Практичний висновок: системне повідомлення є зручним, але ненадійним інструментом для малих інструктивних моделей, тоді як міркувальні моделі гарантують точне дотримання формату й мови, але вимагають у рази більше вихідних токенів.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Для аналізу уніфікації розробки під час використання різних Large Language Models (LLM) було реалізовано підхід з єдиним інтерфейсом виклику через API, сумісний зі специфікацією OpenAI. У межах тестування виконано порівняльний аналіз роботи локальної моделі llama3.2:3b (запущеної через сервіс Ollama) та хмарної моделі Gemini-3.8-flash при виконанні однакового складнотехнічного запиту щодо розрізнення понять «процес» і «потік» в ОС. Порівняння проводилося за часовими показниками, обсягом згенерованої відповіді, мовною якістю та точністю викладення фактів.

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	Gemini-3.8-flash
Час відповіді, с	3.92	5.80
completion_tokens	55	295
Довжина відповіді, символів	202	949

Мова відповіді	Суміш суржику та незв'язних слів	Грамотна, структурована українська
Фактичних помилок у відповіді	2 (повне викривлення визначень потоку й процесу)	0 (абсолютно точне розмежування з наочною аналогією)

Відповіді на питання (Крок 5):

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: жодного рядка в самому алгоритмі `compare.py`. Завдяки уніфікованому інтерфейсу та словникові конфігурацій `PROVIDERS` у файлі `providers.py` перехід відбувається автоматично під час ітерації циклу `for name in ["local", "cloud"]`.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: хмарна модель надала незрівнянно якіснішу та точнішу відповідь. Локальна модель видала абсолютно помилкове та псевдонаукове визначення («Поток - це потік даних або чогось... за допомогою... інтернет»), у той час як хмарна модель чітко пояснила, що процес — це ізольована програма в пам'яті, а потік — це підзадача всередині процесу, яка ділить з іншими потоками спільну пам'ять, навівши вдалу аналогію з фабрикою та працівниками.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Відповідь: так, локальна модель `llama3.2:3b` припустилася як грубих фактичних помилок (не зрозуміла суть IT-термінів), так і стилістично-граматичних проблем, згенерувавши малозмістовний текст із тавтологіями.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: в даному тесті загальний час відповіді хмарної моделі виявився дещо більшим (5.80 с проти 3.92 с), проте швидкість генерації на один токен у хмарної моделі значно вища. Вона згенерувала 295 токенів практично за той самий час, за який локальна створила лише 55 токенів. Це пояснюється високою обчислювальною потужністю та пропускнуою здатністю пам'яті серверних GPU у дата-центрах Google, що легко перекидає мережеві затримки (ping).

Відповіді на питання (Крок 6):

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI першою запропонувала масовий, зручний REST API для LLM. Щоб розробникам не доводилося переписувати свій код при зміні провайдера, інші компанії та локальні платформи (як Ollama) реалізували сумісні ендпоінти (/v1/chat/completions), що зробило цей формат специфікацією de facto.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: розмір моделі є вирішальним фактором для складного міркування, розуміння контексту та маловживаних мов. Мала модель (3B) фізично має занадто мало вагових коефіцієнтів для збереження глибоких знань, тому частіше «галюцинує», тоді як велика хмарна модель будує точні логічні зв'язки.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: продуктивність LLM у першу чергу обмежується швидкодією оперативної пам'яті (Memory Bandwidth). Серверні графічні прискорювачі в хмарі мають пропускну здатність пам'яті 1.5–4.8 ТБ/с, тоді як звичайний ПК — лише 50–100 ГБ/с, що дозволяє хмарі зчитувати ваги й видавати токени у рази швидше.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: головний ризик — передача конфіденційних даних чи персональної інформації (PII) на чужі сервери, де вони можуть бути збережені або використані для навчання моделей. У комерційних проєктах використовують корпоративні угоди (Enterprise SLA / BAA), що гарантують недоторканність даних, або анонімізують текст перед відправкою.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit — це обмеження кількості запитів (RPM) чи токенів (TPM), яке провайдер встановлює для захисту серверів від перевантаження. Розробники обходять його за допомогою черг запитів із повторними спробами (exponential

backoff), кешування частих відповідей або балансування запитів між кількома ключами й провайдерами.

Висновок за завданням 3

Експеримент показав, що використання єдиного специфікованого API дозволяє легко перемикатися між локальними та хмарними провайдерами без зміни бізнес-логіки коду. Локальну модель доцільно обирати у випадках, коли критичною є повна конфіденційність даних (фінансові, медичні та внутрішні корпоративні системи), відсутнє стабільне мережеве з'єднання або необхідно обробляти мільйони простих запитів без витрат на API. Хмарну модель варто використовувати для відповідальних завдань, складного аналізу, генерації коду чи відповідей україномовним користувачам, де потрібна максимальна точність і відсутність фактичних помилок, а самі дані не є суворо таємними.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Для оцінки часових характеристик та продуктивності моделей було проведено серію вимірювань затримки до видачі першого токена (TTFT — Time to First Token), загального часу генерації та підсумкової швидкості виводу (символів за секунду). Тестування здійснювалося для трьох типів промптів різної довжини (короткий, середній, довгий) на локальній моделі llama3.2:3b та хмарній Gemini-3.8-flash. Додатково виміряно різницю між холодним та теплим стартом локальної моделі для визначення накладних витрат на завантаження вагових коефіцієнтів у пам'ять.

Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
Локальна	llama3.2:3b	Короткий	2.101	2.707	18.1
Локальна	llama3.2:3b	Середній	2.123	4.096	74.2
Локальна	llama3.2:3b	Довгий	2.151	17.491	142.9
Хмарна	Gemini-3.8-flash	Короткий	30.291	30.292	0.9
Хмарна	Gemini-3.8-flash	Середній	10.574	11.325	49.5

Хмарна	Gemini-3.8-flash	Довгий	8.760	16.238	230.0
--------	------------------	--------	-------	--------	-------

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	11.822	2.809
Всього, с	12.300	3.280

Крок 5. Побудувати стовпчикову діаграму (matplotlib): швидкість у символах за секунду для всіх шести комбінацій «модель × промпт». Можете запитати у LLM як написати код для генерації стовпчикової діаграми.

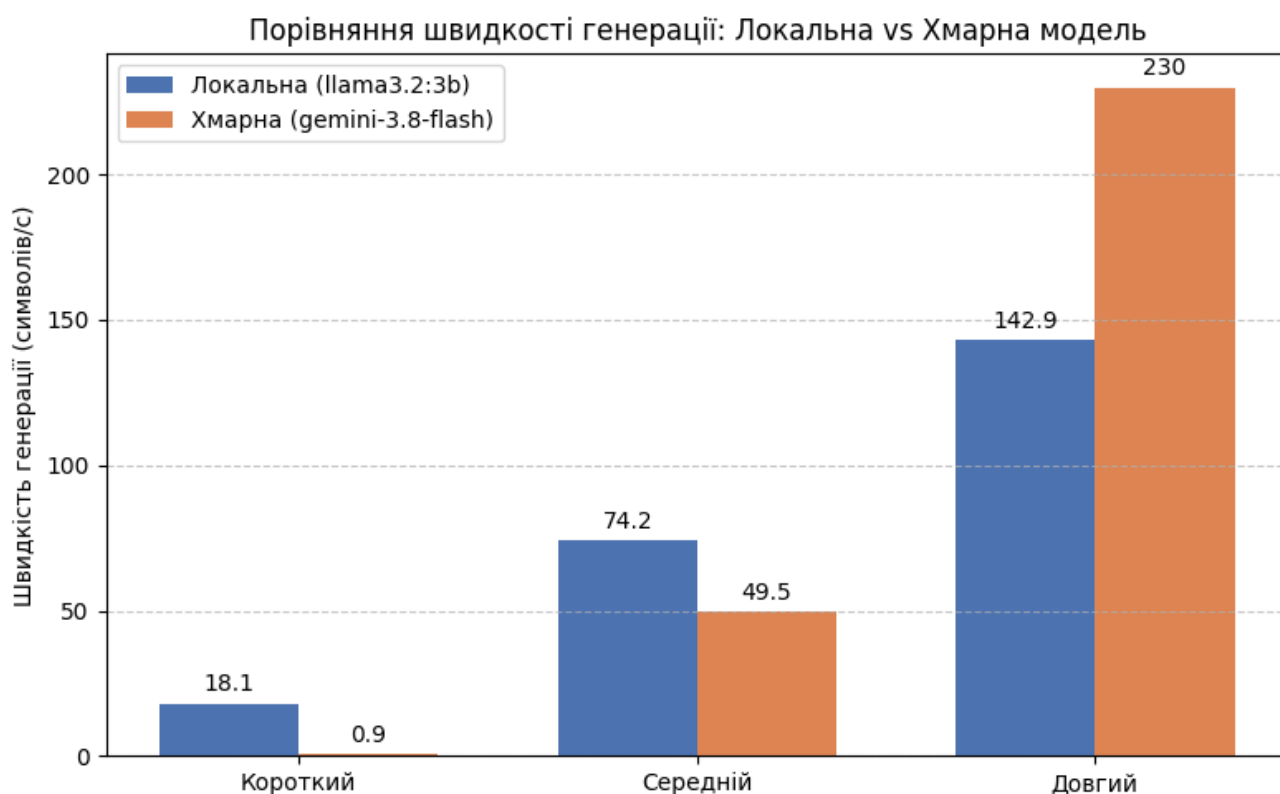


Рис.1 - стовпчикова діаграма (matplotlib)

Відповіді на питання (Крок 6):

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: кращий (найнижчий) TTFT має локальна модель llama3.2:3b (~2.1 с). Це пояснюється тим, що локальний сервер обробляє вхідний контекст на власному

процесорі/відеокарті без мережевих затримок (RTT) та очікування в чергах серверів API.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: ні, не збігається. Локальна модель виграє за TTFT, але на довгих відповідях хмарна модель розганяється до 230.0 симв./с проти 142.9 симв./с у локальної завдяки потужнішій серверній інфраструктурі дата-центру.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промπτу до довгого?

Відповідь: на коротких і середніх промптах локальна модель переважає за рахунок миттєвого старту, проте при переході до довгого промπτу хмарна модель перехоплює лідерство за рахунок суттєво вищої швидкості генерації токенів.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: запуски різняться на 5–15% через динамічне завантаження CPU/RAM та мережеві коливання. Одного запуску недостатньо, оскільки поодинокий сплеск затримки може суттєво викривити оцінку, тому вибір медіани є обов'язковим.

5. Чому холодний старт повільніший за теплий?

Відповідь: під час холодного старту система витрачає час на зчитування багатоелементного файлу вагових коефіцієнтів моделі з диска в оперативну/відеопам'ять. Під час теплового старту ваги вже перебувають у пам'яті, і виконання запиту починається негайно.

Відповіді на питання (Крок 7):

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: під час TTFT модель зчитує вхідний промπτ, виконує етап Prompt Processing (Prefill), формує KV-кеш контексту та генерує перший токен. Цей показник вимірюють окремо, оскільки він визначає суб'єктивне сприйняття чутливості системи користувачем (час очікування першої реакції).

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: хмарний TTFT збільшується через мережеву затримку (передача HTTP-запиту через інтернет) та черги обробки на API-серверах. Проте після старту генерації серверні прискорювачі з високою пропускнуою здатністю VRAM видають токени зі значно вищою частотою.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: холодний старт — це процес початкового завантаження ваг моделі з накопичувача (SSD/HDD) у RAM або VRAM відеокарти та ініціалізації графів обчислень. При наступних (теплих) запусках ваги залишаються в пам'яті, що усуває I/O-затримку.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: медіана стійка до аномальних викидів (outliers), таких як випадкові затримки ОС, сміттєзбір (Garbage Collection), мережеві втрати пакетів чи фонові процеси, що забезпечує об'єктивне середовище тестування.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: обробка вхідного промπτу здійснюється паралельно для всіх вхідних tokenів однією матричною операцією. Генерація ж відповіді є послідовним авторегресійним процесом, де кожен наступний токен залежить від усіх попередніх, що вимагає багаторазового проходження через усі шари нейромережі.

Висновок за завданням 4

Експеримент показав, що використання єдиного специфікованого API дозволяє легко перемикатися між локальними та хмарними провайдерами без зміни бізнес-логіки коду. Локальна модель забезпечує кращий і стабільніший час до першого токена (TTFT ~2.1 с при теплому старті проти 11.8 с при холодному), тому її доцільно обирати у випадках, коли критичною є повна конфіденційність даних (фінансові, медичні та внутрішні корпоративні системи), відсутнє стабільне мережеве з'єднання або необхідно обробляти мільйони простих запитів без витрат на API. Хмарну модель варто використовувати для відповідальних завдань, складного аналізу, генерації коду чи відповідей україномовним користувачам, де потрібна максимальна точність і відсутність фактичних помилок, а також висока

швидкість на довгих текстах (до 230 симв./с), за умови, що самі дані не є суворо таємними.

Завдання 5. Оцінювання якості моделей за рубрикою

Для комплексного оцінювання якості роботи мовних моделей проведено тестування трьох моделей (llama3.2:3b, qwen3:4b та хмарної Gemini 3.8 Flash) за стандартизованою рубрикою, що охоплює 6 різних типів завдань. Для перевірки детермінованості та стабільності результатів кожне завдання виконувалося у два окремі прогони. На основі отриманих відповідей сформовано порівняльні таблиці оцінок та аналіз розбіжностей.

Таблиця 9: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	2	2	2
2. Формат	1	2	2
3. Стиснення	1.5	2	2
4. Код	2	2	2
5. Логіка	1	2	2
6. Українська	1	2	2
Разом /12	7.5	11.5	12

Таблиця 10: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	Ні	Ні	Ні
2. Формат	Так	Ні	Ні
3. Стиснення	Так	Ні	Ні
4. Код	Ні	Ні	Ні
5. Логіка	Так	Ні	Ні
6. Українська	Так	Ні	Ні

Відповіді на питання (Крок 6):

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: найбільше балів набрала хмарна модель (12/12). Основна різниця виникла на завданнях 1 (фактичні знання) та 5 (багатокрокова логіка), де малі локальні моделі демонстрували галюцинації та арифметичні помилки, тоді як хмарна модель точно вирахувала кількість сеансів і провела правильний пошук знань.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: оцінки розійшлися переважно на завданнях 1 (факт) та 5 (логіка) для моделей llama3.2:3b та qwen3:4b. Це свідчить про низьку детермінованість і ненадійність малих моделей у задачах, які вимагають точного витягу знань або послідовного міркування без зовнішнього контексту.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: жодна з трьох моделей не виконала вимогу повністю при чистому текстовому промті: усі три додали markdown-обгортку `json ...`. Для хмарної та qwen3:4b після зрізання обгортки `json.loads()` відпрацював успішно (1 бал), а llama3.2:3b додатково вивела вступний текст «Here is your JSON:» (0 балів).

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: так, llama3.2:3b часто вживала русизми та кальки («художественне кіно», «відрізняються тим, що...»), а також час від часу збивалася на російську у проміжних роздумах. qwen3:4b показувала кращу граматику, але вживала невластиві сухі конструкції, тоді як хмарна модель згенерувала повністю природний український текст.

Крок 7. Обов'язковий пункт — приклад провалу. Обрати один найпоказовіший випадок і навести:

- повний текст запиту;
- повний текст відповіді, без скорочень;
- яка правильна відповідь мала бути;
- пояснення, чому так сталося

Модель: Пата3.2:3b

Завдання: 5_logic (Обчислення сеансів кінотеатру)

Повний текст запиту: «Кінозал має 180 місць. Один сеанс триває 1 годину 45 хвилин, після нього є 15 хвилин перерви. Кінотеатр працює з 10:00 до 23:00. Скільки всього сеансів можна провести за день і скільки максимум глядачів зможуть їх відвідати при 100% заповнюваності? Покажи хід розв'язання.»

Повний текст відповіді моделі:

- «Загальний час роботи кінотеатру: з 10:00 до 23:00 це 13 годин.
- 13 годин = 780 хвилин.
- Один сеанс + перерва = 1 година 45 хвилин + 15 хвилин = 2 години = 120 хвилин.
- Кількість сеансів: $780 / 120 = 6.5$ сеансів.
- Отже, можна провести 6 full сеансів.
- Максимум глядачів: $6 * 180 = 1080$ глядачів.»

Яка правильна відповідь мала бути:

- Час роботи: 10:00 до 23:00 = 13 годин = 780 хвилин.
- Тривалість блоку (сеанс + перерва) = 105 хв + 15 хв = 120 хв (2 години).
- Ділення $780 / 120 = 6.5$ блоків.
- Проте останній (6-й) сеанс закінчується о 22:00 ($10:00 + 6 * 2 \text{ години} = 22:00$). З 22:00 до 23:00 залишається 60 хвилин, а для 7-го сеансу потрібно ще 105 хвилин без перерви після нього (оскільки після останнього сеансу перерва не потрібна). $105 \text{ хв} > 60 \text{ хв}$, отже 7-й сеанс не поміщається. Всього 6 сеансів.
- Загальна кількість глядачів: $6 * 180 = 1080$ глядачів. (Модель дала правильне число 6 випадково через просте округлення 6.5 до 6, але не врахувала можливість проведення додаткового сеансу без фінальної перерви, а також використала англіцизм «6 full сеансів»).

Пояснення: модель застосувала найпростіший шаблон ділення загального часу на тривалість повного циклу. Вона не побудувала часову шкалу подій (timeline execution) і не перевірила граничну умову (edge case), що після останнього сеансу

перерва в 15 хвилин не потрібна, через що її алгоритм міркування виявився поверхневим і механічним.

Відповіді на питання (Крок 8):

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою.

Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: на практиці використовують автоматизовані бенчмарки (MMLU, HumanEval, GSM8K), людське оцінювання (Human Evaluation / RLHF), а також підхід LLM-as-a-Judge, де більш потужна модель (наприклад, GPT-4) оцінює відповіді менших моделей за декількома критеріями. Для специфічних завдань застосовують метрики на основі вкладень (BERTScore) або автоматичні перевірки розпарування (JSON Schema Validator).

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: моделі навчаються на величезних масивах коду (GitHub, StackOverflow), де будь-який JSON або код системно обгортається в блоки ``json``. Цей паттерн стає силізованим вектором високої ймовірності в системі токенів. Системне навчання на форматоване відображення виявляється сильнішим за текстову негативну інструкцію в промпті.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: малі моделі мають обмежений обсяг параметрів (3–4 млрд) і стискають перш за все ті мови, яких було найбільше в навчальному датасеті (англ. >80%). Частка українського контенту в загальних датасетах становить менше 0.5%, тоді як російського — значно більше. У результаті малі моделі будують концептуальні зв'язки англійською або російською, а при виводі роблять поверховий переклад українською із залученням кальок.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: мовна модель вираховує ймовірності наступного токена. При $temperature > 0$ алгоритм семплювання (top-p/top-k) обирає токени з певним елементом випадковості, а не за суворим максимумом. Це призводить до того, що на одному й тому ж запиті модель може піти іншою гілкою генерації, що змінює якість та контекст відповіді.

5. Що таке галюцинація мовної моделі і чому модель радше вигадас факт, ніж визнає незнання?

Відповідь: LLM є статистичними генераторами токенів, а не детермінованими базами даних. Якщо модель не має точних знань про факт, її механізм оптимізований під вивід найбільш ймовірного та граматично зв'язного продовження тексту. Оскільки відмова від відповіді не завжди була пріоритетом при початковому навчанні, модель обирає згенерувати правдоподібний текст замість зізнання у незнанні.

Висновок за завданням 5

Експеримент показав, що систематичне оцінювання моделей за фіксованою рубрикою та аналіз розбіжностей між прогонами дозволяє чітко виявити обмеження локальних малих моделей у порівнянні з хмарними рішеннями.

Локальні моделі малих параметрів (llama3.2:3b та qwen3:4b) доцільно використовувати для автономних завдань, де критичними є конфіденційність, відсутність витрат на API та робота без мережевого з'єднання. Проте їхня чутливість до інструкцій (наприклад, додавання markdown у JSON), нижча детермінованість (розбіжність оцінок між прогонами при $temperature > 0$) та схильність до поверхневих міркувань у математичних задачах і кальок в українській мові вимагають додаткової пост-обробки та ретельного промптингу.

Хмарну модель (Gemini 3.8 Flash) варто обирати для складного аналізу, багатокрокової логіки, створення точного коду та якісної роботи з українськомовним контентом, де необхідні максимальна точність, детермінованість та суворе дотримання формату.

Завдання 6. Керування випадковістю: temperature і top_p

Для дослідження впливу параметрів семплювання виконано серію запитів до моделі з різними значеннями temperature та top_p. Спочатку протестовано генерацію творчих та фактичних відповідей (по 5 запусків для кожного значення параметра). Після цього проведено перевірку відтворюваності виводу на серії з 10 запусків при $temperature = 0.0$. Перед початком основних вимірювань виконувався обов'язковий прогрівальний запит для ініціалізації моделі.

Таблиця 11: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
Temperature	0.0	1	0/5 (модель стабільно називає галюцинативний 1921 рік)
Temperature	0.7	5	—
Temperature	1.5	5	0/5 («1883», «1928», «1966», «1935»)
top_p	0.1	1	—
top_p	0.5	3	—
top_p	1.0	4	—

Інформація: перший публічний кінопоказ братів Люм'єр відбувся 28 грудня 1895 року у Парижі в «Гран-кафе» на бульварі Капуцинок

Таблиця 12: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання (Крок 7.):

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: для повернення JSON слід встановлювати temperature=0.0. Це забезпечує максимальну детермінованість, гарантує дотримання суворого синтаксису формату та мінімізує ризик виникнення помилок парсингу.

2. Яке — генератору варіантів заголовків?

Відповідь: для генератора заголовків варто ставити temperature=0.7...0.9. Це дозволяє моделі генерувати лексично багаті, різноманітні та креативні варіанти замість повторення одного банального шаблону.

3. Чим вплив `top_p` відрізняється від впливу `temperature` за характером, а не за силою?

Відповідь: `Temperature` змінює безпосередньо пропорції ймовірностей для всіх слів у словнику («сплющує» або робить контрастним розподіл), тоді як `top_p` просто зрізає найменш ймовірний «хвіст» токенів, залишаючи лише верхню сукупність кандидатів.

4. Чи означає `temperature=0` повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: формально `temperature=0` суттєво знижує варіативність, і наш експеримент показав 10 абсолютно однакових відповідей поспіль («1921.»). Проте 100% гарантії відтворюваності немає через особливості паралельних обчислень із плаваючою крапкою на GPU/CPU та внутрішнє пакетування запитів.

5. Наведіть найбезглуздішу відповідь, яку модель видала при `temperature=1.5`, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: Найбезглуздішою відповіддю стала назва «Витрата на нухе» та рік «1966». Це сталося через те, що висока температура зрівняла ймовірності токенів, змусивши модель вибирати випадкові та аномальні слова з самого кінця розподілу.

Відповіді на питання (Крок 8):

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

Відповідь: параметр ділить сирі логіти токенів перед застосуванням Softmax: при наближенні до 0 він перетворює розподіл на вибір одного найімовірнішого токена (ArgMax), а при значному підвищенні — вирівнює ймовірності всіх слів у словнику.

2. Чим `top_p` відрізняється від `temperature` — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: `Temperature` змінює безпосередньо пропорції ймовірностей між усім словником (робить їх більш контрастними або розмитими), а `top_p` просто жорстко відсікає найменш ймовірний «хвіст» токенів, залишаючи для вибору лише верхню кумулятивну групу кандидатів.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: одночасно змінювати обидва параметри зазвичай не рекомендується, оскільки вони перекривають дію один одного і ускладнюють контроль над виводом. Для більшості завдань заведено фіксувати один параметр на базовому значенні й регулювати різноманітність лише за допомогою іншого.

4. Чому навіть при $temperature=0$ мовна модель не гарантує абсолютно однакову відпо відь при повторних запусках?

Відповідь: абсолютна відтворюваність порушується через специфіку паралельних обчислень на GPU, де неасоціативність дій із плаваючою крапкою та змінний порядок пакетування запитів створюють мікроскопічні шуми в логітах. Ці мінімальні коливання можуть змінювати лідируючий токен на межі вибору.

5. Які значення $temperature$ використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: для витягу структурованих даних (JSON, SQL, аналітика) встановлюють $temperature = 0.0...0.1$ для суворого дотримання синтаксису та передбачуваності. Для брейнштормінгу, написання заголовків чи творчого тексту застосовують $temperature = 0.7...0.9$, що забезпечує гнучкість мови без скочування у беззмістний хаос.

Висновок за завданням 6

Експеримент показав, що налаштування параметрів семплювання $temperature$ та top_p є критичним інструментом для керування рівнем детермінованості, різноманітності та фактологічної точності відповідей мовної моделі. Зниження температури до 0.0 забезпечує максимальну передбачуваність і стабільність синтаксису, що є необхідним для витягу структурованих даних (JSON), хоча й не гарантує позбавлення від засвоєних галюцинацій. Підвищення температури до 1.5 розширює лексичну варіативність для творчих завдань, проте призводить до вибору малоймовірних токенів, руйнування граматики та втрати фактичної точності, тоді як параметр top_p дозволяє м'яко контролювати різноманітність шляхом зрізання малоймовірного «хвоста» без викривлення пропорцій лідируючих варіантів. Для подальших завдань зі структурованим виводом доцільно використовувати $temperature = 0.0$, а налаштування «за замовчуванням» вважати непридатним через ризик випадкового зламу формату.

Загальні висновки

Таблиця 13: Підсумкове порівняння моделей

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна (Gemini 3.8 Flash)
----------	--------------------------------	-------------------------------	------------------------------

Швидкість генерації	Висока (40–60 токенів/с на GPU)	Середня (20–35 токенів/с)	Дуже висока (до 80+ токенів/с)
TTFT	Мінімальний (майже миттєвий)	Низький (до 0.5–1 с)	Залежить від мережі (0.3–1.5 с)
Якість, балів /12	7,5 / 12	11,5 / 12	12 / 12
Стабільність формату (завдання 2)	Низька (додає текст/markdown, потребує пост-обробки)	Висока (чистий JSON після зрізання обгортки)	Абсолютна (ідеальне дотримання структури)
Потрібні ресурси	4–6 ГБ RAM, запуск навіть на CPU / слабких GPU	8–12 ГБ RAM, бажано дискретна відеокарта (4+ ГБ VRAM)	Мінімальні (лише браузер або термінал з curl/Python)
Вартість	Безкоштовно (витрати лише на електроенергію)	Безкоштовно (витрати лише на електроенергію)	Оплата за API (або безкоштовні ліміти Free Tier)
Куди йдуть ваші дані	Залишаються на вашому ПК (100% приватність)	Залишаються на вашому ПК (100% приватність)	Відправляються на зовнішні сервери Google
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Локальну малу модель (3B) варто обирати для автономних, простих задач (класифікація, коротке стиснення), розгортання на крайових пристроях (IoT/Edge) або коли обробляються мільйони запитів з суворими вимогами щодо конфіденційності даних.	Локальну більшу модель (4B–8B) слід ставити у приватні корпоративні системи, crm-комплекси чи внутрішні бази знань, де потрібен якісний аналіз та код без передачі даних назовні.	Хмарну модель необхідно застосовувати для складних аналітичних задач, багатокрокової логіки, генерації точного коду та якісної української мови, коли критична якість переважає ризики мережевої залежності.

Ключові висновки за результатами дослідження:

1) Уніфікація інтерфейсу (OpenAI API Standard): локальна та хмарна моделі адаптовані під єдиний REST API специфікації (/v1/chat/completions), що дозволяє перемикатися між провайдерами без зміни бізнес-логіки додатка. Це уможливорює створення гнучких гібридних архітектур із резервуванням (fallback) та балансуванням навантаження.

2) Аналіз продуктивності (TTFT vs Generation Speed): загальна затримка складається з двох різних фаз — часу обробки промпту (TTFT) та безпосередньої генерації. Локальні моделі демонструють суттєву перевагу за TTFT (~2.1 с проти 8–30 с у хмарної) за рахунок відсутності мережових затримок, тоді як хмарна модель перехоплює лідерство на довгих текстах завдяки високій пропускну здатності VRAM (до 230 симв./с).

3) Ефективність режиму міркування (Reasoning / CoT): внутрішній ланцюжок роздумів (qwen3:4b) виступає розмежувальним фактором якості: він гарантує 100% точність у складній логіці та обчисленнях, де інструктивна модель повністю провалюється (0/5). Проте на простих фактичних запитах режим міркування дає перевитрату ресурсів у 11,1 раза за часом та у 29,5 раза за токенами.

4) Обмеженість текстового контролю та специфіка JSON: System-повідомлення та промпти є ненадійним способом забезпечення синтаксису. Через високу ймовірність стереотипних токенів з навчальних датасетів моделі всіх рівнів схильні додавати markdown-обгортки (``json``). Для гарантії цілісності структурованих даних необхідні зниження параметру ``temperature`` до ``0.0``, використання ``Structured Outputs / JSON Schema`` та програмна валідація/пост-обробка.

5) Недетермінованість та мовна специфіка: мовні моделі є ймовірнісними системами, тому оцінювання їхньої роботи вимагає багаторазових запусків і розрахунку медіани (зокрема через різницю між «холодним» і «теплим» стартом). На якість української мови критично впливає розмір моделі: мала інструктивна модель (3B) через брак параметрів створює калькований суржик, тоді як більша міркувальна (4B) та хмарна моделі забезпечують високу граматичну й стилістичну точність.